

MboaTrust AI Deployment and CI/CD Explained

1. The Big Picture

MboaTrust AI is now arranged like a small production system, not just one folder running on your laptop.

It has five important parts:

1. Mobile app: the Android app installed on a user's phone.
2. GitHub repository: the central place where the project code lives.
3. GitHub Actions: the automation engine that checks and builds the app.
4. Expo EAS Build: the cloud builder that creates the installable APK.
5. Render backend services: the public API services and database the app talks to.

The phone does not talk to your laptop anymore after deployment. It talks to public HTTPS URLs on Render.

2. Current MboaTrust Structure

The project root is:

```
C:\Users\ngong\Documents\CUB\MboaTrust
```

Important folders:

```
apps/mobile
```

This is the Expo React Native mobile app. It contains the screens, navigation, mobile service clients, Android project, eas.json, and app configuration.

```
services/api
```

This is the main FastAPI backend. It handles authentication, registration, KYC records, business records, receipts, trust score endpoints, statistics, preferences, notifications, and database persistence.

```
services/data-collection
```

This contains the data collection agent and the MTN/Orange provider simulator. The app requests mobile money data through the API/data agent. The simulator is useful for demo and testing.

```
services/credit-scoring
```

This contains the credit scoring API and the trained model files used to generate the trust score.

```
.github/workflows
```

This contains GitHub Actions workflows. These workflows define the CI/CD pipeline.

```
render.yaml
```

This is the Render Blueprint file. It tells Render which backend services and database to create.

3. What Git Does

Git is the version control system.

It records snapshots of the project. Every time you run:

```
git add .  
git commit -m "message"  
git push
```

you are sending a new project version to GitHub.

Git does not deploy by itself. GitHub and other tools react to Git changes.

4. What GitHub Does

GitHub is the remote home of the code.

Your repository is:

```
https://github.com/Duval1703/MboaTrust-AI
```

GitHub stores the code, tracks changes, and runs automation through GitHub Actions.

When you push code to GitHub, two kinds of things can happen:

1. GitHub Actions can run tests or builds.
2. Render can detect the new commit and redeploy backend services.

5. What GitHub Actions Does

GitHub Actions is the CI/CD automation tool.

CI means Continuous Integration. It checks that new code is valid before you trust it.

CD means Continuous Delivery or Continuous Deployment. It prepares or deploys the app automatically.

Your project currently has two mobile workflows:

```
.github/workflows/mobile-ci.yml
```

This runs when mobile code changes are pushed to main or develop, or when a pull request touches the mobile app. It installs dependencies and runs:

```
npm run typecheck
```

This helps catch TypeScript errors before building an APK.

```
.github/workflows/mobile-android-build.yml
```

This builds the Android app with Expo EAS. It can be triggered manually from GitHub Actions using Run workflow, or automatically when you push a tag like:

```
mobile-v1.0.1
```

The workflow:

1. Downloads the repository.
2. Installs Node.
3. Authenticates to Expo using `EXPO\_TOKEN`.
4. Installs mobile dependencies.
5. Runs TypeScript checking.
6. Calls Expo EAS Build.
7. EAS creates the APK.

6. What Expo EAS Build Does

Expo EAS Build is the cloud build service for Expo and React Native apps.

Instead of building the APK only on your laptop, Expo builds it on their servers.

Your build profile is defined in:

```
apps/mobile/eas.json
```

The important profile right now is:

```
preview
```

It creates an internal Android APK that can be installed directly on a phone.

The preview build now uses:

```
EXPO_PUBLIC_API_URL=https://mboatrust-api.onrender.com
EXPO_PUBLIC_DATA_AGENT_URL=https://mboatrust-data-collection.onrender.com
```

That means the APK knows where the public backend lives.

7. What Render Does

Render hosts the backend services on the internet.

The Render Blueprint creates:

```
mboatrust-api
mboatrust-data-collection
mboatrust-provider-simulator
mboatrust-credit-scoring
mboatrust-db
```

Each service has a public URL.

The mobile app calls the main API:

```
https://mboatrust-api.onrender.com
```

The API then calls the other services when needed:

```
https://mboatrust-data-collection.onrender.com
https://mboatrust-credit-scoring.onrender.com
```

The data collection agent talks to the provider simulator:

```
https://mboatrust-provider-simulator.onrender.com
```

8. What Docker Does

Docker packages each backend service with the Python environment it needs.

This avoids the problem of "it works on my laptop but not on the server."

Each service has a Dockerfile:

```
services/api/Dockerfile
services/data-collection/Dockerfile
services/data-collection/Dockerfile.provider
services/credit-scoring/Dockerfile
```

Render reads these Dockerfiles and builds deployable containers.

9. What PostgreSQL Does

PostgreSQL is the real database.

It stores records such as:

1. Users and merchant registration data.
2. KYC status.
3. Sales records.
4. Expense records.
5. Stock records.
6. Receipt verification history.
7. Trust score records.
8. Preferences and profile data.

The API connects to the database using DATABASE_URL.

On Render, DATABASE_URL is provided automatically from the mboatrust-db database.

10. How All Pieces Interact

Here is the runtime flow after deployment:

```
User Phone
|
| opens installed APK
v
Mboatrust Mobile App
|
| HTTPS requests
v
mboatrust-api on Render
|
| stores and reads app data
v
mboatrust-db PostgreSQL
```

For trust score:

```
Mobile App
|
v
mboatrust-api
|
| requests transaction data
v
mboatrust-data-collection
|
v
mboatrust-provider-simulator
|
```

```
v
mboatrust-data-collection
|
v
mboatrust-api
|
| asks AI model for score
v
mboatrust-credit-scoring
|
v
mboatrust-api
|
| saves result
v
mboatrust-db
```

For APK delivery:

```
Developer pushes code to GitHub
|
v
GitHub Actions
|
v
Expo EAS Build
|
v
APK link in Expo dashboard
|
v
User downloads and installs APK
```

11. What Happens When You Push Changes to GitHub

There are three different kinds of changes.

Backend Changes

Example:

```
services/api/main.py
services/data-collection/cub_data_agent.py
services/credit-scoring/src/scoring_agent.py
```

If Render auto-deploy is enabled, pushing backend changes to GitHub can redeploy the backend automatically.

Users do not need to reinstall the APK for most backend-only changes. The installed app calls the backend each time it loads data.

Example:

If you fix a bug in the API endpoint that calculates dashboard totals, users may see the fix the next time the app calls that endpoint.

Mobile JavaScript/UI Changes

Example:

```
apps/mobile/src/screens/Dashboard/HomeScreen.tsx  
apps/mobile/src/screens/Records/AddSaleScreen.tsx
```

With the current setup, these changes do not automatically update the app already installed on a phone.

To deliver these changes right now:

1. Push the changes.
2. Run the Android EAS Build workflow.
3. Download/install the new APK.
4. Share the new APK link with users.

Native Mobile Changes

Example:

1. Adding a new Expo native module.
2. Changing Android permissions.
3. Editing files inside `apps/mobile/android`.
4. Changing camera, SMS, or secure-store native behavior.

These always require a new APK build.

Even with over-the-air updates, native changes cannot be safely delivered as a simple JavaScript update.

12. Will Installed Apps Update Automatically?

Short answer: not yet.

Right now, if someone downloads the APK and installs it:

1. Backend changes can affect them automatically because the app talks to the backend.
2. Mobile app code changes will not automatically appear on their phone.
3. Native changes definitely require a new APK.

To make JavaScript and UI changes update automatically, we need to configure Expo EAS Update.

13. What EAS Update Would Add

EAS Update is Expo's over-the-air update system.

It can send JavaScript, styling, and asset changes to installed apps without building a new APK.

However, it has limits:

1. It cannot update native Android code.
2. It cannot add new native modules to an old APK.
3. It must match the app runtime version.
4. It should be configured carefully with channels like `preview` and `production`.

If we add EAS Update, then a future flow could look like this:

```
Developer changes only JS/UI code
|
v
Push to GitHub
|
v
GitHub Actions runs tests
|
v
GitHub Actions publishes EAS Update
|
v
Installed apps receive update
```

But your current app is not configured for this yet.

14. The Current CI/CD Pipeline

Current pipeline:

```
Push to GitHub
|
v
Mobile CI runs typecheck
```

For APK:

```
Manual GitHub Actions run
|
v
Android EAS Build workflow
|
v
Expo creates APK
|
v
You share/install APK
```


For backend:

```
Push to GitHub
|
v
Render reads render.yaml
|
v
Render rebuilds changed backend services
|
v
Public API updates
```

15. Recommended Next CI/CD Improvements

For a stronger deployment process, we should add these in stages:

1. Backend CI:
 - Python syntax check.
 - API import check.
 - Optional endpoint tests.
2. Backend deployment verification:
 - After Render deploys, automatically call `/health` endpoints.
3. Mobile release automation:
 - Build APK when a tag like `mobile-v1.0.2` is pushed.
 - Keep manual preview builds for testing.
4. EAS Update:
 - Configure OTA updates.
 - Add a workflow that publishes JS/UI updates to preview.
 - Later add a controlled production channel.
5. Production app distribution:
 - For public users, publish through Google Play.
 - APK links are okay for testing, demos, and pilots, but Google Play is better for trust, updates, and security.

16. What To Remember

The backend is a living online system. If it changes on Render, installed apps can immediately benefit.

The APK is a snapshot installed on the phone. If the phone app code changes, users need either:

1. A new APK, or
2. EAS Update configured for compatible JavaScript-only changes.

The CI/CD pipeline is the bridge between your code and what users receive.

GitHub stores the truth. GitHub Actions checks and builds it. Expo creates the mobile package. Render runs the backend. The phone app consumes the public backend.

17. Official References

- ? GitHub Actions workflows: <https://docs.github.com/en/actions/concepts/workflows-and-actions/workflows>
- ? Expo EAS Build: <https://docs.expo.dev/build/introduction/>
- ? Expo internal distribution builds: <https://docs.expo.dev/tutorial/eas/internal-distribution-builds>
- ? Expo EAS Update: <https://docs.expo.dev/eas-update/getting-started/>
- ? Render free instances: <https://render.com/docs/free>
- ? Render Blueprint specification: <https://render.com/docs/blueprint-spec>